

A nivel de ingeniería de sistemas móviles, el micrófono es un sensor de transducción que convierte ondas mecánicas de presión acústica en señales eléctricas y, posteriormente, en datos digitales binarios. En esta sesión enseñaremos a gestionar el hardware de captura de audio, entender conceptos de compresión de audio y manejar flujos de datos en tiempo real mediante el almacenamiento de archivos.

Para este tutorial utilizaremos el paquete **record**, que es actualmente el estándar moderno, más ligero y mantenido para la grabación de audio en Flutter, sustituyendo a librerías obsoletas y ofreciendo soporte nativo completo para Android e iOS.

Tutorial: Procesamiento de Señales Acústicas y Captura de Audio (Micrófono)

Este módulo práctico enseña a tomar el control del hardware del micrófono del dispositivo, gestionar los permisos dinámicos de grabación de audio, configurar contenedores de codificación digital y almacenar archivos de audio en el almacenamiento temporal del sistema operativo.

1. Fundamentos Teóricos y Conceptos del Paquete **record**

El desarrollo con captura de audio requiere comprender cómo los sistemas operativos procesan las señales analógicas del entorno.

- **AudioEncoder (Codificación de Audio):** Es el algoritmo encargado de comprimir y digitalizar la señal de audio cruda proveniente del transductor. El paquete provee diferentes enums de codificación:
- **AudioEncoder.aacLc:** (Advanced Audio Coding - Low Complexity). Es el estándar de la industria móvil. Ofrece una alta calidad de audio con archivos muy ligeros. Ideal para streaming o notas de voz comunes.
- **AudioEncoder.pcm16bit:** Captura el audio sin compresión (onda pura). Genera archivos masivos pero es el formato obligatorio si planean integrar la app con APIs de Inteligencia Artificial como *Whisper de OpenAI* para transcripción de voz a texto.
- **Gestión de Rutas Temporales:** Al grabar audio, los datos se escriben directamente en el almacenamiento a una velocidad crítica. Enseñaremos a utilizar el directorio temporal del sistema operativo (**Cache**), garantizando que si el usuario graba múltiples audios de prueba, el espacio se libere automáticamente cuando el teléfono requiera mantenimiento de memoria.
- **La Máquina de Estados de Grabación:** El ciclo de vida del hardware del micrófono se controla mediante estados lógicos asíncronos (**isRecording**, **isPaused**, **stop**). Al igual que con la cámara, el micrófono debe cerrarse de forma estricta para no dejar el canal de escucha del hardware abierto.

2. Configuración del Entorno (Plataforma Nativa)

La privacidad del micrófono es máxima. Los sistemas operativos bloquean de forma agresiva cualquier intento de inicialización del micrófono si no está explícitamente justificado.

En Android (`android/app/src/main/AndroidManifest.xml`)

Añade la siguiente línea de permiso de hardware justo antes de la etiqueta de apertura `<application>`:

```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

En iOS (`ios/Runner/Info.plist`)

Añade la llave de seguridad que le explicará al alumno la razón de la captura al momento de desplegar el Pop-up de privacidad:

```
<key>NSMicrophoneUsageDescription</key>
<string>Necesitamos acceso al micrófono para la práctica de captura y
almacenamiento de audio.</string>
```

Estructura de Carpetas del Módulo

Agregamos la última característica (`feature`) de forma limpia en el árbol de nuestro proyecto:

```
mi_app_sensores/
├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── core/
│   │   └── theme/
│   │       └── theme.dart
│   └── features/
│       ├── home/
│       ├── camera/
│       ├── gps/
│       ├── wifi/
│       └── microphone/          <-- CARPETA DEL MÓDULO ACTUAL
│           ├── screens/
│           └── microphone_screen.dart
```

Asegúrate de agregar la dependencia oficial en el archivo `pubspec.yaml` junto con `path_provider` (necesaria para localizar las rutas de almacenamiento temporal del móvil):

```
dependencies:
  flutter:
    sdk: flutter
```

```
record: ^5.1.2      # O la versión estable más reciente
path_provider: ^2.1.3 # Necesario para gestionar directorios de archivos
```

3. Código Fuente Completo y Comentado

Archivo `lib/features/microphone/screens/microphone_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:record/record.dart';
import 'package:path_provider/path_provider.dart';
import 'dart:io';

class MicrophoneScreen extends StatefulWidget {
  const MicrophoneScreen({super.key});

  @override
  State<MicrophoneScreen> createState() => _MicrophoneScreenState();
}

class _MicrophoneScreenState extends State<MicrophoneScreen> {
  // El controlador principal del ciclo de vida del micrófono
  late final AudioRecorder _audioRecorder;

  bool _isRecording = false;
  String _statusText = "Micrófono listo para grabar";
  String? _lastRecordedPath;

  @override
  void initState() {
    super.initState();
    // Instanciamos el grabador al iniciar la pantalla
    _audioRecorder = AudioRecorder();
  }

  // 1. Control de flujo: Iniciar la captura de audio
  Future<void> _startRecording() async {
    try {
      // Validar y solicitar el permiso nativo de grabación en tiempo de ejecución
      if (await _audioRecorder.hasPermission()) {

        // Obtener el directorio temporal (Cache) del dispositivo (Android/iOS)
        final directory = await getTemporaryDirectory();

        // Definimos el nombre del archivo con una marca de tiempo para evitar
        colisiones
        final String filePath =
          '${directory.path}/audio_cache_${DateTime.now().millisecondsSinceEpoch}.m4a';

        // Configuración de los parámetros de ingeniería del hardware de audio
```

```

    const RecordConfig config = RecordConfig(
      encoder: AudioEncoder.aacLc, // Codificación estándar y ligera
      bitRate: 128000,             // 128 kbps (Calidad de audio excelente)
      sampleRate: 44100,           // 44.1 kHz (Frecuencia de muestreo
estándar CD)
    );

    // Encendemos el hardware y comenzamos a escribir en disco de forma
asíncrona
    await _audioRecorder.start(config, path: filePath);

    setState(() {
      _isRecording = true;
      _lastRecordedPath = filePath;
      _statusText = "🔴 GRABANDO AUDIO EN TIEMPO REAL...";
    });
  } else {
    setState(() {
      _statusText = "Permiso de micrófono denegado por el usuario.";
    });
  }
} catch (e) {
  setState(() {
    _statusText = "Error al inicializar el hardware: $e";
  });
}
}

// 2. Control de flujo: Detener la captura de audio
Future<void> _stopRecording() async {
  try {
    // Al detener, el paquete cierra el archivo y retorna la ruta exacta donde
se guardó
    final path = await _audioRecorder.stop();

    setState(() {
      _isRecording = false;
      _statusText = "Audio guardado con éxito.";
    });

    if (path != null) {
      // Validamos el tamaño del archivo binario generado para comprobar que
tiene datos
      final file = File(path);
      final int fileBytes = await file.length();
      final double fileKiloBytes = fileBytes / 1024;

      setState(() {
        _statusText = "✅ Grabación finalizada.\n"
          "Ruta: ...${path.substring(path.length - 30)}\n"
          "Tamaño: ${fileKiloBytes.toStringAsFixed(1)} KB";
      });
    }
  } catch (e) {

```

```

        setState(() {
          _statusText = "Error al detener la grabación: $e";
        });
      }
    }

    @override
    void dispose() {
      // CRUCIAL: Destruir la instancia del grabador al salir de la pantalla
      // Esto garantiza liberar el hardware y apagar el daemon de grabación del S.O.
      _audioRecorder.dispose();
      super.dispose();
    }

    @override
    Widget build(BuildContext context) {
      return Scaffold(
        appBar: AppBar(title: const Text('Sensor: Captura de Audio')),
        body: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              // Consola de Estado del Hardware
              Card(
                elevation: 4,
                color: _isRecording ? Colors.red.withOpacity(0.05) :
Colors.blue.withOpacity(0.05),
                child: Padding(
                  padding: const EdgeInsets.all(24.0),
                  child: Column(
                    children: [
                      Icon(
                        _isRecording ? Icons.mic : Icons.mic_none,
                        size: 64,
                        color: _isRecording ? Colors.red : Colors.blue,
                      ),
                      const SizedBox(height: 16),
                      Text(
                        _statusText,
                        textAlign: TextAlign.center,
                        style: const TextStyle(fontFamily: 'monospace', fontSize:
14, fontWeight: FontWeight.bold),
                      ),
                    ],
                  ),
                ),
              const SizedBox(height: 40),

              // Controles de Acción Centralizados
              Row(
                mainAxisAlignment: MainAxisAlignment.center,

```

```

        children: [
          if (!_isRecording)
            FloatingActionButton.large(
              onPressed: _startRecording,
              backgroundColor: Colors.blueAccent,
              foregroundColor: Colors.white,
              child: const Icon(Icons.play_arrow),
            ),
          if (_isRecording)
            FloatingActionButton.large(
              onPressed: _stopRecording,
              backgroundColor: Colors.redAccent,
              foregroundColor: Colors.white,
              child: const Icon(Icons.stop),
            ),
        ],
      ),
      const SizedBox(height: 20),
      const Text(
        'Nota: El archivo se almacena en el directorio temporal del sistema operativo, ideal para procesamiento y optimización de memoria.',
        textAlign: TextAlign.center,
        style: TextStyle(fontSize: 11, color: Colors.grey),
      ),
    ],
  ),
),
);
}
}

```

Vinculación Final en el Menú Principal

Para culminar el laboratorio completo de sensores, abre `lib/features/home/screens/home_screen.dart` y activa el último botón del GridView reemplazando la tarjeta del Micrófono:

```

// 1. Agrega el import correspondiente al inicio de home_screen.dart:
import '../..'/microphone/screens/microphone_screen.dart';

// 2. Modifica la tarjeta del Micrófono dentro del GridView:
_SensorCard(
  title: 'Micrófono / Audio',
  icon: Icons.mic,
  color: Colors.red, // Cambiamos de gris a rojo indicando la activación del módulo
  onTap: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const MicrophoneScreen()),
    );
  },
)

```

```
);
},
),
```

💡 Guía Didáctica

1. **Muestreo y Bitrate (Conceptos de Telecomunicaciones):** Los parámetros `sampleRate: 44100` y `bitRate: 128000`. Estos valores determinan la fidelidad de la digitalización matemática de la onda. Si están haciendo una app de notas de voz simple, pueden bajar el bitrate a `64000` (64 kbps) para generar archivos la mitad de pesados sin perder claridad de voz.
2. **Validación de Datos en Disco:** Revisen la consola después de grabar un audio corto y uno largo. Verán cómo el cálculo de los kilobytes cambia dinámicamente. Esto refuerza la comprensión de la persistencia de archivos en sistemas operativos móviles.
3. **Seguridad Pasiva:** Qué pasaría si no se invoca `_audioRecorder.dispose()`. Al salir de la pantalla, el sistema operativo mantendrá el icono de "Micrófono en uso" en la barra de estado superior del teléfono, lo que genera desconfianza en el usuario e incluso puede causar el baneo de la app en las tiendas de Google/Apple por políticas de privacidad.

El código está utilizando `getTemporaryDirectory()`, lo que guarda el archivo dentro del **directorio de caché privado de la aplicación** (una ruta parecida a `/data/user/0/com.example.miapp/cache/...`). Por seguridad, **ningún explorador de archivos comercial ni el usuario común tienen permiso para ver esa carpeta** (a menos que el teléfono esté *rooteado*). Está diseñado así para que el sistema operativo borre esos archivos automáticamente cuando el teléfono necesite espacio.

Solución: Modificar el código para guardarlo en la carpeta pública "Download"

Para ello, primero necesitamos modificar un permiso en el archivo `android/app/src/main/AndroidManifest.xml` (justo abajo del permiso del micrófono) para permitir la escritura en el almacenamiento compartido:

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
android:maxSdkVersion="29" />
```

Y en tu archivo `microphone_screen.dart`, vamos a cambiar el método `_startRecording()` reemplazando `getTemporaryDirectory()` por `getExternalStorageDirectory()` y apuntando a la carpeta pública.

```
Future<void> _startRecording() async {
  try {
    if (await _audioRecorder.hasPermission()) {

      // CAMBIO AQUÍ: Usamos el almacenamiento externo del dispositivo
      final directory = await getExternalStorageDirectory();
```

```

if (directory != null) {
  // Buscamos la ruta base quitando los subdirectorios privados de Android
  // para poder meterlo en la carpeta raíz de descargas del Xiaomi
  String pathDestino = directory.path.split('/Android')[0] + '/Download';

  // Creamos el archivo dentro de la carpeta Descargas (Download)
  final String filePath =
    '$pathDestino/Prueba_Audio_${DateTime.now().millisecondsSinceEpoch}.m4a';

  const RecordConfig config = RecordConfig(
    encoder: AudioEncoder.aacLc,
    bitRate: 128000,
    sampleRate: 44100,
  );

  await _audioRecorder.start(config, path: filePath);

  setState(() {
    _isRecording = true;
    _lastRecordedPath = filePath;
    _statusText = "● GRABANDO EN CARPETA DOWNLOADS...";
  });
} else {
  setState(() {
    _statusText = "Permiso de micrófono denegado.";
  });
}
} catch (e) {
  setState(() {
    _statusText = "Error al inicializar el hardware: $e";
  });
}
}

```

fusión de manifiestos (*Manifest Merger Conflict*) en Android. Ocurre porque la dependencia interna de la cámara (`camera_android_camerax`) también declara el permiso `WRITE_EXTERNAL_STORAGE` pero acotado hasta la versión 28 (`maxSdkVersion="28"`), mientras que nuestro código solicita que se mantenga hasta la versión 29 (`maxSdkVersion="29"`).

Cuando el compilador de Android (Gradle) intenta unificar ambos archivos en uno solo antes de empaquetar el APK, encuentra esta discrepancia de versiones y se detiene para evitar inconsistencias de hardware.

Sigue estos dos sencillos pasos en tu archivo `AndroidManifest.xml` para solucionarlo:

Paso 1: Modificar el archivo `android/app/src/main/AndroidManifest.xml`

Abre el archivo y realiza dos cambios en la etiqueta principal `<manifest>` y en el permiso en conflicto:

1. Agrega el espacio de nombres de herramientas (`xmlns:tools`) en la etiqueta superior `<manifest>`.
2. Agrega la instrucción `tools:replace="android:maxSdkVersion"` en el permiso de almacenamiento.

El archivo debe quedar estructurado de la siguiente manera:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
          xmlns:tools="http://schemas.android.com/tools"> <!-- 1. AGREGA ESTA
LÍNEA AQUÍ -->

    <!-- Tus demás permisos (cámara, gps, etc.) -->
    <uses-permission android:name="android.permission.RECORD_AUDIO" />

    <!-- 2. REEMPLAZA TU PERMISO DE ALMACENAMIENTO POR ESTE EXACTAMENTE: -->
    <uses-permission
        android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="29"
        tools:replace="android:maxSdkVersion" />

    <application
        android:label="mi_app_sensores"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <!-- ... resto del archivo ... -->
```

Paso 2: Limpieza obligatoria antes de compilar

Como Gradle dejó a medias el árbol de compilación debido al error, es mandatorio limpiar los archivos temporales para que aplique la regla de reemplazo desde cero. Ejecuta esto en la terminal de tu proyecto:

```
flutter clean
flutter pub get
flutter run
```